BÁO CÁO TIẾN ĐỘ HỌC TẬP

Linux, Virtualization → Container

Tổng kết kiến thức Tuần 1 và Tuần 2

Linux | Network | Storage | Permission | Hypervisor | Container

Người báo cáo: **Lê Tấn Lực** · **Đỗ Hoàng Hiếu**

TUẦN 1

Linux, Virtualization và hạ tầng nền tảng

Tài nguyên máy chủ | Network | Storage & LVM | User & Permission | systemd & Firewall | Hypervisor & VM

Mục tiêu Tuần 1 và tiêu chí đầu ra

#	Chủ đề	Tiêu chí đầu ra (theo roadmap)
1	Linux doanh nghiệp: CPU, RAM, disk, process	Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác
2	Network: IP, routing, DNS, kết nối VM	Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab
3	Storage: partition, LVM, filesystem, mount	Tự thêm được disk mới, tạo LVM và mount vào hệ thống
4	User, group và permission	Tạo và phân quyền được theo đúng yêu cầu của ứng dụng
5	Service (systemd) và firewall	Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu
6	Hypervisor và vòng đời VM	Thao tác đầy đủ vòng đời VM trên hypervisor đang dùng

Cách tiếp cận: đo → xác định nút thắt (bottleneck) → phân bổ hoặc giới hạn.

Môi trường Lab: 2 VM `vm-web` (nginx) và `vm-db` (mariadb) trên KVM / VirtualBox.

Mỗi chủ đề gồm: lý thuyết, lệnh thực hành và một ví dụ dễ hiểu để trình bày.

CPU và Load Average

Cấu trúc CPU

- **Socket → Core → Thread.** Mỗi thread là 1 **CPU logic** (`nproc`)
- **CPU time:** `us` ứng dụng · `sy` kernel · `wa` chờ disk · `st` bị VM khác lấy

Load average (1 · 5 · 15 phút)

- Số process đang chạy hoặc chờ CPU/I/O
- So với số CPU logic: **load = số CPU** là đầy tải, **gấp đôi** là quá tải

```
nproc ; uptime        # số CPU · load average
top                   # nhấn 1 để xem từng CPU
mpstat -P ALL 1 3     # thống kê từng core
```

Chỉ số cao	Nguyên nhân → Xử lý
`us`	App tính toán nặng → tối ưu, thêm vCPU
`sy`	Nhiều system call → kiểm tra thread
`wa`	Chờ disk → điều tra disk, không phải thiếu CPU
`st`	Host quá tải → migrate VM

Bộ nhớ (RAM, swap) và Disk

```
free -h
#               total        used        free      buff/cache   available
# Mem:           3.8Gi       1.1Gi       210Mi       2.5Gi       2.4Gi
vmstat 1    # si/so > 0 liên tục = đang swap → thiếu RAM
```

RAM

- Linux dùng RAM rảnh làm **page cache**, nên *free* thấp là bình thường. Chỉ số cần nhìn là **available**
- **RSS** = RAM vật lý đang dùng · **VSZ** = không gian ảo (thường lớn, không đáng lo)
- **Swap** chậm hơn RAM hàng nghìn lần. Với DB server nên đặt `vm.swappiness` = 1–10
- Hết RAM và swap → **OOM Killer** giết process (`dmesg -T | grep -i oom`)

Disk: hai câu hỏi khác nhau

Còn bao nhiêu chỗ? (capacity) và **disk có đang nghẽn không?** (I/O)

```
df -hT ; df -i          # hết inode cũng "đầy disk"
du -sh /var/* | sort -h # thư mục chiếm chỗ
iostat -xz 1 3          # %util ~100%, await cao = nghẽn
sudo iotop -o           # process nào đang đọc/ghi
lsof +L1                # file đã xoá nhưng còn bị giữ
```

Chỉ số iostat	Ý nghĩa
`r/s` , `w/s`	Số IOPS đọc/ghi mỗi giây
`r_await` , `w_await`	Độ trễ (ms). SSD nên dưới vài ms
`%util`	~100% liên tục = disk bão hòa

Process, giới hạn tài nguyên và phương pháp USE

Process

- Trạng thái: `R` chạy · `S` ngủ · `D` chờ I/O · `T` dừng · `Z` zombie
- Signal: **SIGTERM 15** dừng lịch sự · **SIGKILL 9** buộc dừng · **SIGHUP 1** reload cấu hình
- `nice` từ −20 (ưu tiên cao) đến 19. Giảm nice cần quyền root
- **cgroups** giới hạn CPU/RAM/IO cho nhóm process. Đây là nền tảng của systemd và container

```
ps -eo pid,ppid,user,stat,%cpu,%mem,cmd --sort=-%cpu | head
kill -15 1234 ; pkill -HUP nginx
nice -n 10 tar czf backup.tgz /data
ionice -c3 -p 1234              # I/O ưu tiên "idle"
sudo systemctl set-property nginx.service \
     CPUQuota=50% MemoryMax=512M
```

USE Method: với mỗi tài nguyên, kiểm tra **U**tilization (mức dùng), **S**aturation (hàng đợi) và **E**rrors (lỗi). Quy trình 60 giây: `uptime` → `dmesg -T | tail` → `vmstat` → `mpstat` → `pidstat` → `iostat` → `free` → `sar -n DEV`

💡 **Ví dụ trình bày: server như một nhà hàng**

CPU là đầu bếp, **load** là số món đang chờ nấu: 4 đầu bếp mà 8 món chờ là quá tải gấp đôi. **RAM** là mặt bàn bếp, **cache** là nguyên liệu bày sẵn, dọn đi được ngay. **%wa** cao là đầu bếp đứng chờ người lấy đồ trong kho (disk).

🎯 **Đầu ra:** Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác

Địa chỉ IP, CIDR và routing

CIDR	Mask	Host dùng được	Ví dụ
/8	255.0.0.0	16.777.214	10.0.0.0/8 (private)
/16	255.255.0.0	65.534	172.16.0.0/12
/24	255.255.255.0	254	192.168.56.0/24
/28	255.255.255.240	14	Subnet nhỏ cho DMZ
/32	255.255.255.255	1	Route tới 1 IP

- **Số host = 2^(32 − prefix) − 2** (trừ địa chỉ network và broadcast)
- `192.168.56.10/24` → network `.0`, broadcast `.255`, host từ `.1` đến `.254`
- **Private (RFC 1918)** không ra Internet được, phải đi qua NAT
- Cùng subnet thì gửi thẳng qua switch (ARP lấy MAC). Khác subnet thì phải qua gateway

Gói tin đi đâu?

- Kernel chọn route **khớp dài nhất** (longest prefix match). Không khớp route nào thì đi theo **default route** `0.0.0.0/0`
- Có nhiều route cùng độ dài thì route có **metric** thấp hơn được ưu tiên
- **Lỗi hay gặp:** 2 interface cùng có default gateway → traffic ra sai cổng. Chỉ nên có 1 default route

```
ip -br addr                 # interface + IP gọn
ip route
# default via 10.0.2.2 dev enp0s3 metric 100
# 192.168.56.0/24 dev enp0s8 src 192.168.56.10
ip route get 8.8.8.8        # kernel chọn route nào
ip route add 10.10.0.0/16 via 192.168.56.1
sysctl -w net.ipv4.ip_forward=1   # Linux làm router
```

Cấu hình IP vĩnh viễn và DNS

```
# RHEL/Rocky - NetworkManager
nmcli con mod enp0s8 ipv4.method manual \
  ipv4.addresses 192.168.56.10/24 ipv4.gateway 192.168.56.1 \
  ipv4.dns "8.8.8.8 1.1.1.1"
nmcli con up enp0s8
```

```
# Ubuntu - Netplan: /etc/netplan/01-lab.yaml
network:
  version: 2
  ethernets:
    enp0s8:
      addresses: [192.168.56.10/24]
      nameservers: { addresses: [8.8.8.8] }
# netplan try  → tự rollback sau 120s nếu mất kết nối
```

Lệnh `ip addr add` chỉ có tác dụng tạm thời và mất khi reboot. Muốn cấu hình vĩnh viễn phải dùng nmcli hoặc Netplan.

DNS phân giải tên

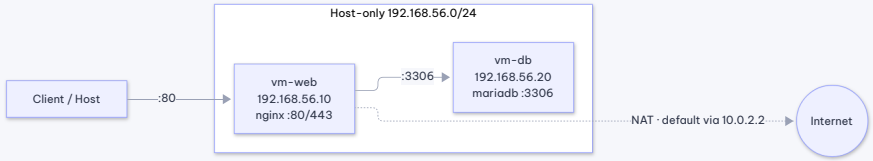
- Thứ tự phân giải ghi trong `/etc/nsswitch.conf` (`hosts: files dns`): đọc `/etc/hosts` trước, sau đó mới hỏi DNS server trong `/etc/resolv.conf`
- **Luồng truy vấn:** client → resolver (8.8.8.8) → root → TLD (`.com`) → authoritative. Kết quả được cache theo **TTL**
- **Bản ghi:** A · AAAA · CNAME · MX · NS · PTR · TXT
- Ubuntu dùng **systemd-resolved**: resolv.conf trỏ tới `127.0.0.53`, xem cấu hình thật bằng `resolvectl status`

```
dig google.com +short ; dig -x 8.8.8.8
dig @1.1.1.1 example.com MX
getent hosts vm-db       # đi qua nsswitch (cả /etc/hosts)
# /etc/hosts trong lab
192.168.56.10  vm-web
192.168.56.20  vm-db
```

Mạng ảo cho VM và sơ đồ Lab

Chế độ	Cách hoạt động	Internet	Host↔VM	VM↔VM
NAT	Hypervisor làm router NAT	✅	Port fwd	Tùy
Bridged	Như máy thật trong LAN	✅	✅	✅
Host-only	Mạng riêng host + VM	❌	✅	✅
Internal	Chỉ VM với VM	❌	❌	✅

KVM/libvirt: `virbr0` (NAT 192.168.122.0/24, DHCP dnsmasq), bridge `br0` cho bridged · **VMware:** VMnet8 NAT, VMnet1 Host-only, VMnet0 Bridged · **Proxmox:** Linux bridge `vmbr0`



Luồng: Client → `vm-web:80` qua mạng host-only → `vm-db:3306`. Riêng `vm-web` ra Internet qua NAT (NIC `enp0s3`, 10.0.2.15).

💡 **Ví dụ trình bày: mạng như hệ thống bưu điện**

IP là địa chỉ nhà, **subnet /24** là cùng một con phố nên gửi thư thì đưa tận tay (switch/ARP). **Gateway** là bưu cục đầu phố, **routing table** là bảng chỉ đường chọn tuyến cụ thể nhất. **DNS** là danh bạ, tra tên "vm-db" ra 192.168.56.20 trước khi gửi.

🎯 **Đầu ra:** Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab

Chuỗi lưu trữ, partition và LVM



	MBR (msdos)	GPT
Disk tối đa	2 TiB	~9.4 ZB (gần như không giới hạn)
Số partition	4 primary (hoặc 3 + extended)	128 mặc định
Boot	BIOS legacy	UEFI
Độ bền	1 bảng duy nhất	Có bản sao header ở cuối disk + CRC

```
lsblk -f ; blkid ; findmnt       # xem thiết bị, UUID, mount
parted /dev/sdb --script mklabel gpt \
  mkpart primary 0% 100% set 1 lvm on
```

Thành phần	Là gì	Lệnh
PV Physical Volume	Disk/partition được khởi tạo cho LVM	`pvcreate`, `pvs`
VG Volume Group	"Bể" dung lượng gộp từ nhiều PV, chia thành PE (4 MiB)	`vgcreate`, `vgextend`
LV Logical Volume	"Partition ảo" cắt từ VG, nơi tạo filesystem	`lvcreate`, `lvextend`

Vì sao dùng LVM: có một lớp trừu tượng giữa disk và filesystem, nhờ đó gộp được nhiều disk, **mở rộng khi đang chạy** và tạo snapshot.

Thực hành LVM, filesystem và /etc/fstab

```
pvcreate /dev/sdb1
vgcreate vg_data /dev/sdb1
lvcreate -n lv_app -L 5G vg_data      # hoặc -l 100%FREE
mkfs.xfs /dev/vg_data/lv_app
# Mở rộng khi thêm disk /dev/sdc - không downtime
pvcreate /dev/sdc && vgextend vg_data /dev/sdc
lvextend -r -L +3G /dev/vg_data/lv_app   # -r: resize FS
# Snapshot LV trước khi nâng cấp
lvcreate -s -n snap_app -L 1G /dev/vg_data/lv_app
```

```
echo 'UUID=<uuid> /data/app xfs defaults,nofail 0 0' >> /etc/fstab
mount -a && findmnt --verify     # LUÔN kiểm tra trước reboot
```

💡 **Ví dụ trình bày: storage như kho hàng**

Disk là mảnh đất, **partition** là chia lô, **LVM** gộp nhiều lô thành một khu (VG) rồi chia lại linh hoạt thành các kho (LV). **Filesystem** là cách xếp kệ, **mount** là gắn biển địa chỉ "/data/app". Sắp đầy thì thêm disk, chạy vgextend + lvextend -r, không downtime.

Tiêu chí	ext4	xfs
Mặc định trên	Ubuntu, Debian	RHEL, Rocky, Alma
Mở rộng online	`resize2fs`	`xfs_growfs`
Thu nhỏ	Được (phải umount)	**Không hỗ trợ**
Điểm mạnh	Ổn định, linh hoạt	File lớn, song song cao

6 cột của fstab

thiết bị · mount point · loại FS · options (`defaults` , `noatime` , `nofail` , `ro`) · dump · fsck pass (0 tắt, 1 cho `/` , 2 cho FS khác)

- Dùng **UUID** vì tên `/dev/sdX` có thể đổi khi thêm disk
- Một dòng sai có thể khiến máy vào **emergency mode**. `nofail` tránh lỗi này với disk phụ

🎯 **Đầu ra:** Tự thêm được disk mới, tạo LVM và mount vào hệ thống

User, group và quyền cơ bản rwx

File	Nội dung
`/etc/passwd`	user:x:UID:GID:mô tả:home:shell
`/etc/shadow`	Mật khẩu đã hash, chính sách hết hạn (chỉ root đọc)
`/etc/group`	group:x:GID:danh sách thành viên
`/etc/sudoers.d/`	Quy tắc sudo, sửa bằng `visudo`

```
useradd -m -s /bin/bash -c "Dev Luc" luc ; passwd luc
groupadd devteam
usermod -aG devteam luc     # thiếu -a sẽ XOÁ các group cũ!
useradd -r -s /usr/sbin/nologin -d /opt/myapp myapp
chage -M 90 luc ; usermod -L luc ; userdel -r olduser
```

UID 0 là root. UID dưới 1000 là system user dành cho dịch vụ (nginx, mysql), không có shell đăng nhập.

Quyền	Số	Với file	Với thư mục
r	4	Đọc nội dung	Liệt kê tên file (`ls`)
w	2	Sửa nội dung	Tạo / xoá / đổi tên file bên trong
x	1	Thực thi	Đi vào (`cd`), truy cập file bên trong

3 quy tắc hay bị nhầm

1. Kernel kiểm tra theo thứ tự **owner** → **group** → **others**, áp dụng nhóm đầu tiên khớp, **không cộng dồn**
2. Đọc `/a/b/file` cần quyền `x` trên `/a` và `/a/b`. Kiểm tra bằng `namei -l`
3. **Xoá file** phụ thuộc quyền `w` của **thư mục**, không phụ thuộc quyền của file

```
chmod 640 config.yml ; chmod -R g+rX /srv/share
chown -R myapp:devteam /opt/myapp
umask      # 0022 → file mới 644, thư mục mới 755
```

Quyền đặc biệt, ACL và sudo

Bit	Số	Ý nghĩa	Ví dụ
SUID	4000	File chạy với quyền của owner	`/usr/bin/passwd`
SGID	2000	File mới trong thư mục kế thừa group	Thư mục dự án chung
Sticky	1000	Chỉ owner mới xoá được file của mình	`/tmp` (drwxrwxrwt)

```
chmod 2775 /srv/project                  # SGID thư mục nhóm
find / -perm -4000 -type f 2>/dev/null   # audit file SUID
setfacl -m u:deploy:rx /opt/myapp        # ACL cho 1 user
setfacl -d -m g:devteam:rwX /srv/project # default ACL
getfacl /opt/myapp
```

```
# visudo -f /etc/sudoers.d/devteam
%devteam ALL=(root) /usr/bin/systemctl restart myapp, \
                    /usr/bin/journalctl -u myapp
deploy   ALL=(myapp) NOPASSWD: /opt/myapp/bin/deploy.sh
sudo -l -U luc        # xem user được phép chạy gì
```

Nguyên tắc quyền tối thiểu

- Không đăng nhập trực tiếp bằng root. Tắt `PermitRootLogin` trong sshd_config
- Mỗi ứng dụng một system user riêng, không có shell, chỉ sở hữu thư mục của nó
- **SELinux** (RHEL): rwx đúng mà vẫn bị từ chối thì kiểm tra `ls -Z`, `ausearch -m avc`, `restorecon -Rv`

💡 **Ví dụ trình bày: phân quyền như thẻ ra vào toà nhà**

User là nhân viên, **group** là phòng ban, **rwx** là được xem, sửa, đi vào. Chỉ user `myapp` đọc được file mật khẩu DB (600), dev chỉ đọc log, deploy ghi code qua ACL. **sudo giới hạn** là thẻ chỉ mở đúng phòng cần thiết, không phải chìa khoá vạn năng.

🎯 **Đầu ra:** Tạo và phân quyền được theo đúng yêu cầu của ứng dụng

systemd và vòng đời dịch vụ

systemd là PID 1 trên hầu hết bản Linux hiện đại. Nó khởi động hệ thống, quản lý dịch vụ, log (journald), mount và timer. Đơn vị quản lý là **unit**: `.service` `.socket` `.timer` `.mount` `.target`

Lệnh `systemctl …`	Tác dụng
`start` / `stop` / `restart`	Chạy / dừng / khởi động lại ngay
`reload`	Đọc lại cấu hình, không ngắt kết nối
`enable --now`	Tự chạy khi boot và chạy ngay
`disable` / `mask`	Không tự chạy / cấm hoàn toàn
`status` · `list-units --failed`	Trạng thái, PID, log gần nhất · dịch vụ lỗi
`daemon-reload`	**Bắt buộc** sau khi sửa unit file

```
systemctl edit nginx       # tạo override thay vì sửa file gốc
systemd-analyze blame      # dịch vụ nào làm boot chậm
journalctl -u myapp -f ; journalctl -b -1   # log boot trước
```

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=myapp
WorkingDirectory=/opt/myapp
EnvironmentFile=/etc/myapp/app.env
ExecStart=/opt/myapp/bin/server --port 8080
Restart=on-failure
RestartSec=5
MemoryMax=512M
CPUQuota=50%
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

Firewall và kiểm tra port từ đầu đến cuối

```
# firewalld (RHEL/Rocky) - quản lý theo zone
firewall-cmd --get-active-zones ; firewall-cmd --list-all
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --add-rich-rule='rule family=ipv4
  source address=192.168.56.10/32 port port=3306
  protocol=tcp accept'      # chỉ vm-web được vào DB
firewall-cmd --reload       # permanent có hiệu lực sau reload
```

```
# ufw (Ubuntu)
ufw default deny incoming ; ufw default allow outgoing
ufw allow OpenSSH           # LUÔN mở SSH trước khi enable!
ufw allow 80/tcp
ufw allow from 192.168.56.10 to any port 3306
ufw enable ; ufw status numbered ; ufw delete 3
# mức thấp: nft list ruleset ; iptables -L -n -v
```

Kiểm tra port end-to-end

`ss -tulpn` (dịch vụ có LISTEN không, trên 0.0.0.0 hay 127.0.0.1?) → `nc -zv 192.168.56.20 3306` từ máy khác → `curl -I http://192.168.56.10`

- Dịch vụ chỉ listen **127.0.0.1** thì máy khác không vào được dù firewall đã mở. Phải sửa `bind-address`
- RHEL dùng port lạ thì cần mở cả SELinux: `semanage port -a -t http_port_t -p tcp 8081`

Ví dụ trình bày: systemd là quản lý ca, firewall là bảo vệ cổng

systemd cho dịch vụ vào ca khi boot (**enable**), tự gọi lại khi dịch vụ "ngã" (**Restart=on-failure**) và ghi sổ nhật ký (**journalctl**). Firewall mặc định đóng mọi cửa, chỉ mở 80/443 cho khách. Cửa 3306 của DB chỉ mở cho đúng vm-web, nhờ vậy bề mặt tấn công nhỏ nhất.

Đầu ra: Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu

Hypervisor là gì?

Hypervisor (VMM) chia tài nguyên vật lý thành nhiều máy ảo độc lập. CPU hiện đại hỗ trợ ảo hóa phần cứng (**Intel VT-x / AMD-V**, kiểm tra bằng `grep -cE "vmx|svm" /proc/cpuinfo`) để VM chạy gần tốc độ thật.

	Type 1 (bare-metal)	Type 2 (hosted)
Chạy trên	Trực tiếp phần cứng	Một OS có sẵn
Ví dụ	ESXi, Proxmox VE (KVM), Hyper-V, Xen, Nutanix AHV	VirtualBox, VMware Workstation/Fusion
Dùng cho	Production, datacenter	Lab cá nhân, dev

KVM · QEMU · libvirt

- **KVM**: module kernel biến Linux thành hypervisor
- **QEMU**: giả lập thiết bị cho VM
- **libvirt**: lớp quản lý (`virsh`, `virt-install`, `virt-manager`)
- Proxmox và OpenStack đều dựa trên KVM

Thiết bị paravirtual

`virtio-blk/scsi` cho disk, `virtio-net` cho NIC, hiệu năng tốt hơn thiết bị giả lập. VMware dùng VMware Tools, `vmxnet3`, `pvscsi`.

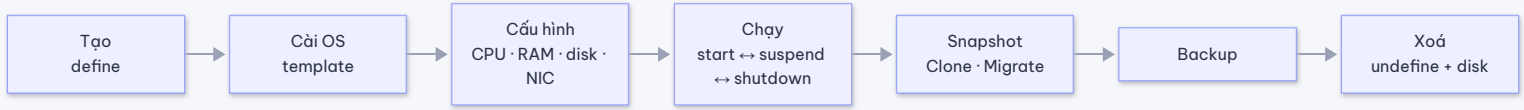
Overcommit

Cấp tổng vCPU/RAM cho các VM vượt tài nguyên thật. **vCPU 3–4:1** thường chấp nhận được. **RAM overcommit rủi ro hơn** (balloon, host phải swap).

Định dạng disk

qcow2 (KVM, thin + snapshot) · **raw** (nhanh nhất) · **vmdk** (VMware) · **vhdx** (Hyper-V). Thin provisioning tiết kiệm chỗ nhưng có thể làm đầy datastore.

Vòng đời VM và thao tác trên các nền tảng




```
virt-install --name vm-web --memory 2048 --vcpus 2 \
  --disk size=20,format=qcow2,bus=virtio \
  --cdrom /iso/Rocky-9.iso --os-variant rocky9 \
  --network network=default,model=virtio
virsh list --all ; virsh dominfo vm-web
virsh start | shutdown | suspend | resume vm-web
virsh setvcpus vm-web 4 --config ; virsh setmem vm-web 4G --config
virsh attach-disk vm-web data.qcow2 vdb --persistent
virt-clone --original vm-web --name vm-web2 --auto-clone
virsh undefine vm-web2 --remove-all-storage
```

Thao tác	Proxmox (`qm`)	VMware ESXi	VirtualBox
Tạo	`qm create 101 --memory 2048 --cores 2`	Wizard / PowerCLI `New-VM`	`createvm` + `modifyvm`
Start / Stop	`qm start` / `shutdown 101`	Power On / Shut Down	`startvm --type headless`
Snapshot	`qm snapshot 101 pre-upd`	Take Snapshot	`snapshot vm take`
Backup	`vzdump 101 --mode snapshot`	Veeam, vSphere Data Protection	`export vm -o vm.ova`

Snapshot và Backup không phải là một

	Snapshot	Backup
Bản chất	Điểm khôi phục, lưu phần chênh lệch (delta) so với disk gốc	Bản sao độc lập, đầy đủ của dữ liệu
Vị trí	Cùng storage với VM	Storage khác, tốt nhất ở site khác
Mất storage gốc	**Mất luôn** snapshot	**Vẫn khôi phục được**
Dùng khi	Trước khi update, rollback nhanh	Phòng thảm họa, lưu trữ lâu dài
Giữ lâu	Không nên (làm chậm I/O, phình disk)	Theo chính sách retention

```
virsh snapshot-create-as vm-web pre-update "Truoc khi update kernel"
virsh snapshot-revert vm-web pre-update
virsh dumpxml vm-web > /backup/vm-web.xml      # backup cold
qemu-img convert -O qcow2 -c vm-web.qcow2 /backup/vm-web.qcow2
```

Quy tắc 3-2-1: 3 bản dữ liệu, trên 2 loại phương tiện, 1 bản ở ngoài site.

Application-consistent: với DB, dùng qemu-guest-agent (`virsh domfsfreeze`) hoặc dump DB trước khi snapshot để tránh dữ liệu ghi dở.

RPO / RTO: mất tối đa bao nhiêu dữ liệu / khôi phục trong bao lâu. Backup chưa từng restore thử thì coi như chưa có backup.

💡 **Ví dụ: hypervisor như toà chung cư**

Hypervisor là ban quản lý chia căn hộ (VM). **Snapshot** là chụp ảnh căn hộ trước khi sửa, nhưng ảnh vẫn nằm trong toà nhà. **Backup** là bản sao cất ở toà nhà khác, cháy toà nhà vẫn khôi phục được.

🎯 **Đầu ra:** Thao tác đầy đủ vòng đời VM trên hypervisor đang dùng

Troubleshooting nhanh

Triệu chứng	Kiểm tra	Hướng xử lý
Server chậm	`uptime`, `top`, `vmstat`, `iostat`	Xác định CPU / RAM / IO, xử lý process nguyên nhân
Boot vào emergency mode	`journalctl -xb`, `/etc/fstab`	Sửa dòng fstab sai, thêm `nofail`
Disk đầy	`df -h`, `df -i`, `du`, `lsof +L1`	Dọn log (logrotate), restart process giữ file, mở rộng LV
Service không chạy	`systemctl status`, `journalctl -u`	Sửa config hoặc quyền, `daemon-reload`
Không truy cập được port	`ss -tulpn`, `firewall-cmd --list-all`, `nc -zv`	Sửa bind address, mở firewall, SELinux port
Permission denied	`id`, `namei -l`, `getfacl`, `ls -Z`	Sửa owner / mode / ACL, `restorecon`
VM không có mạng	`ip a`, `ip r`, `virsh domiflist`, `virsh net-list`	Kiểm tra NIC model, bridge, DHCP, default route

Tài liệu tham khảo: RHEL Documentation · Ubuntu Server Guide · Brendan Gregg – USE Method · *The Linux Command Line*, *How Linux Works* · libvirt.org, Proxmox VE Admin Guide. Chứng chỉ phù hợp: **RHCSA (EX200)**, **LFCS**.

TUẦN 2

Container

Kiến trúc | Image & Registry | Dockerfile | Network | Storage | Compose

Container là gì? So sánh với VM

Container là tiến trình được cô lập

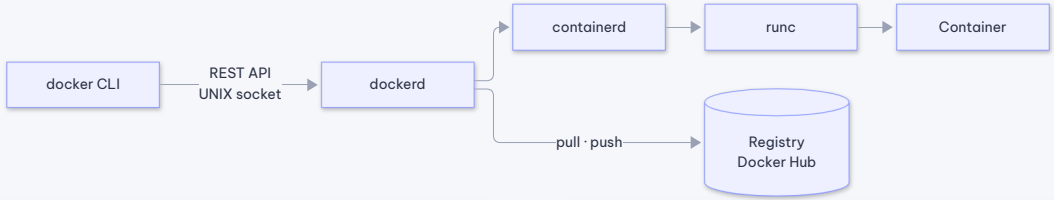
- Docker đóng gói và chạy ứng dụng trong một môi trường cô lập "lỏng"
- Container dùng chung **kernel host**, VM chạy **kernel khách** riêng → mật độ workload cao hơn
- **Namespaces** cô lập những gì tiến trình *nhìn thấy*
- **cgroups** giới hạn tài nguyên được *dùng*. RHEL thêm **SELinux** để bảo mật
- **Union FS**: image gồm nhiều layer chỉ đọc xếp chồng

	Virtual Machine	Container
Ảo hóa	Phần cứng (hypervisor)	Hệ điều hành (kernel)
Kernel	Mỗi VM một kernel	Dùng chung kernel host
Kích thước · khởi động	Vài GB · phút	Vài chục MB · giây

Namespace	Cô lập
`PID`	ID tiến trình (container có PID 1 riêng)
`Network`	Interface, IP, port, route
`Mount`	Các điểm mount
`UTS`	Hostname, tên miền NIS
`IPC`	Hàng đợi tin nhắn, shared memory
`User`	UID/GID (root trong container ≠ root host)
`Cgroup`	Cây cgroup nhìn thấy
`Time`	Đồng hồ boot / monotonic

Từ Linux 3.8, tạo **user namespace** không cần đặc quyền. Các loại khác thường cần `CAP_SYS_ADMIN`. Đây là nền tảng của chế độ **rootless**.

OCI và kiến trúc Docker



3 đặc tả OCI

- **Image Spec:** manifest, index, các layer và config
- **Runtime Spec:** chạy một "filesystem bundle". Luồng: tải image → giải nén thành bundle → runtime chạy
- **Distribution Spec:** API phân phối image, chuẩn hóa từ Docker Registry HTTP API V2
- Image và Distribution bản 1.1.0 (15/02/2024). **Podman, Buildah, Skopeo** tương thích OCI

Thành phần Docker

- Mô hình **client-server**. CLI gọi daemon qua REST API
- **dockerd** quản lý image, container, network, volume
- **containerd** lo vòng đời container, mặc định gọi **runc**
- **Docker Desktop** (Mac/Win/Linux, kèm Compose…) khác **Docker Engine** (`docker-ce`). Lab RHEL 9 dùng Engine

💡 **Ví dụ trình bày: VM là nhà riêng, container là căn hộ**

VM là nhà riêng, có móng riêng (kernel riêng). **Container** là căn hộ chung một móng (kernel host) nhưng có cửa khóa riêng (**namespace**) và công tơ điện nước riêng (**cgroups**). Nhẹ hơn và dọn vào nhanh hơn.

🎯 **Đầu ra:** Giải thích được cơ chế container và khác biệt so với VM trong 5 phút

Image = layers + config + manifest

Cấu trúc image (OCI)

- Gói chuẩn chứa file, binary, thư viện và cấu hình. Hai nguyên tắc: **bất biến** và **ghép từ layer**
- **Manifest** trỏ tới layer bằng **digest** (định danh theo nội dung). **Config** chứa lệnh chạy, biến môi trường
- **Multi-arch:** image index (Docker gọi là *manifest list*) trỏ tới một manifest cho mỗi nền tảng
- Sau khi tải, image được kiểm tra bằng hash

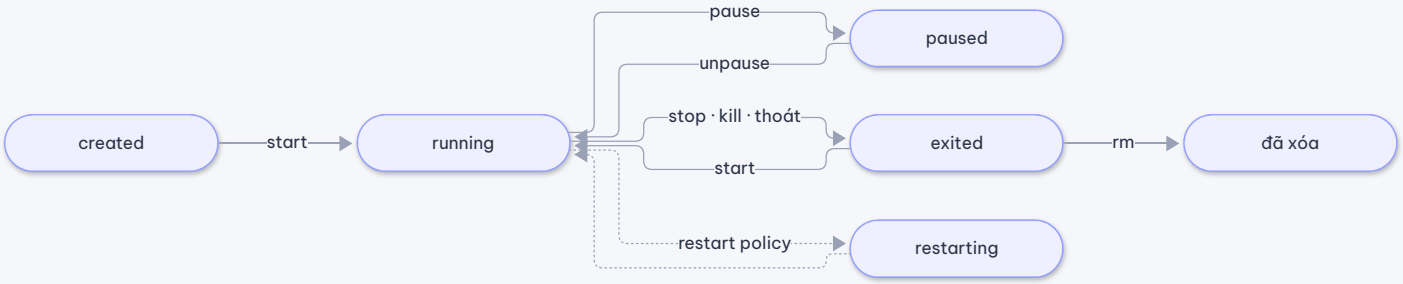
	Tag `:1.4.2` , `:latest`	Digest `@sha256:…`
Bản chất	Con trỏ, có thể bị gán lại	Bất biến theo nội dung
Dùng khi	Dev, thử nghiệm	**Production**, cần tái lập

```
docker pull nginx                  # không tag → :latest
docker pull nginx@sha256:<digest> # ghim đúng một bản
docker image ls                    # kích thước ĐÃ giải nén
docker image history nginx         # từng layer và lệnh tạo ra nó
docker image inspect nginx -f '{{.RepoDigests}}'
docker buildx imagetools inspect nginx   # các nền tảng multi-arch
```

💡 **Ví dụ trình bày: image là khuôn, container là bánh**

Một khuôn đúc được nhiều bánh, và sửa một chiếc bánh không làm đổi khuôn. **Tag** là nhãn "khuôn mới nhất", có thể dán sang khuôn khác. **Digest** là số seri khắc trên khuôn, không đổi được.

Vòng đời container và các lệnh



Lệnh	Chức năng
`run`	`create` + `start`
`create` / `start`	Tạo container chưa chạy / chạy container đã tạo
`stop`	Dừng có ân hạn: SIGTERM, hết hạn mới SIGKILL
`kill`	Gửi SIGKILL ngay (mặc định)
`rm`	Xóa container (`-f` nếu đang chạy)
`exec`	Chạy lệnh trong container đang chạy
`logs` · `inspect`	Xem log · chi tiết dạng JSON
`ps -a` · `top` · `stats`	Danh sách (cả đã dừng) · tiến trình · tài nguyên

```
docker run -d --name web -p 8080:80 \
  --memory 512m --cpus 1 --restart unless-stopped nginx:1.27
docker ps -a --filter status=exited
docker exec -it web sh
docker logs -f --tail 50 web
docker stop web ; docker start web ; docker rm -f web
```

Trạng thái: created · running · paused · restarting · exited · dead.
Resource limit: vượt `--memory` thì container bị **OOM kill**. `--cpus 1` giới hạn tối đa 1 core. Không đặt limit thì một container có thể chiếm hết tài nguyên host.

Registry và quản lý image

Registry

- Tên đầy đủ: `registry[:port]/namespace/repo:tag`. `docker pull fedora` thực chất là `docker.io/library/fedora:latest`
- Tag có `host:port` ở đầu thì Docker push tới registry đó
- `docker login` lưu credential ở `~/.docker/config.json` (Podman/skopeo: `$XDG_RUNTIME_DIR/containers/auth.json`)
- Registry riêng `registry:3` chỉ để test. Production cần **TLS** và kiểm soát truy cập

Docker Hub	Giới hạn pull
Chưa đăng nhập	100 lượt / 6 giờ cho mỗi IPv4 (hoặc subnet IPv6 /64)
Personal	200 lượt / 6 giờ
Pro / Team / Business	Không giới hạn

```
docker run -d -p 5000:5000 --name registry registry:3
docker tag webapp:1.4.2 localhost:5000/team/webapp:1.4.2
docker push localhost:5000/team/webapp:1.4.2
docker image prune -a    # xóa image không dùng
docker system df         # xem dung lượng đã dùng
docker system prune      # dọn container dừng, cache…
```

Lệnh	Đối tượng	Giữ được gì
`save` / `load`	Image	Các layer, tag và metadata
`export` / `import`	Filesystem container	Một lớp phẳng, mất lịch sử và config

Bị chặn **abuse limit** thì nhận lỗi 429 ngắn gọn. Hết **pull limit** thì nhận thông báo dài kèm link tài liệu. skopeo đọc được file `docker save` (định dạng docker-archive).

🎯 **Đầu ra:** Pull, tag, push image lên registry; quản lý được vòng đời container và dung lượng image

Các lệnh Dockerfile

Lệnh	Tác dụng
`FROM`	Bắt đầu một stage, chọn base image. Nhiều FROM là multi-stage. Bỏ tag thì dùng `latest`
`RUN`	Chạy lệnh và tạo layer mới. Cache không tự hết hạn, dùng `--no-cache` để chạy lại
`COPY` / `ADD`	COPY sao chép từ context, stage hoặc image. ADD thêm tải URL/Git, tự giải nén tar, `--checksum`
`WORKDIR`	Thư mục làm việc, tự tạo nếu chưa có. Dùng đường dẫn tuyệt đối thay cho `RUN cd`
`ARG` / `ENV`	ARG chỉ có lúc build (`--build-arg`), ENV nằm luôn trong image
`EXPOSE`	**Chỉ là tài liệu.** Muốn publish phải dùng `-p` / `-P`
`USER`	User/UID mặc định cho RUN, CMD, ENTRYPOINT phía sau
`VOLUME`	Mount point cho dữ liệu gắn ngoài
`HEALTHCHECK`	starting → healthy / unhealthy (mặc định 30s, thử 3 lần)
`LABEL` · `ONBUILD`	Metadata key-value · trigger chạy khi image được dùng làm base

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
      -r requirements.txt
COPY . .
USER 10001
EXPOSE 8000
HEALTHCHECK CMD curl -f \
  http://localhost:8000/health || exit 1
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

Không dùng ARG/ENV cho bí mật: giá trị hiện trong `docker history`, và unset ENV ở layer sau không xóa được. **Exec form** `["a","b"]` phải dùng nháy kép và không thay biến `$HOME`. **Shell form** chạy qua `/bin/sh -c`.

CMD và ENTRYPOINT

	Không ENTRYPOINT	ENTRYPOINT shell: `entry p1`	ENTRYPOINT exec: `["entry","p1"]`
Không CMD	Lỗi, không cho phép	`/bin/sh -c entry p1`	`entry p1`
CMD `["cmd","p1"]`	`cmd p1`	`/bin/sh -c entry p1`	`entry p1 cmd p1`
CMD `cmd p1`	`/bin/sh -c cmd p1`	`/bin/sh -c entry p1`	`entry p1 /bin/sh -c cmd p1`

Quy tắc dùng

- **ENTRYPOINT** là lệnh chính, **CMD** là tham số mặc định. `docker run image <args>` ghi đè CMD
- ENTRYPOINT dạng shell **bỏ qua CMD** và không nhận tín hiệu, nên tránh dùng
- Đặt ENTRYPOINT sẽ **reset CMD** kế thừa từ base image
- CMD gần như luôn dùng exec form. Script entrypoint kết thúc bằng `exec "$@"` để app thành **PID 1** và nhận SIGTERM

```
# Dockerfile
ENTRYPOINT ["/docker-entrypoint.sh"]
CMD ["nginx", "-g", "daemon off;"]
# docker-entrypoint.sh
set -e
envsubst < /tpl/app.conf > /etc/app.conf
exec "$@"            # thay shell bằng CMD → PID 1
# Khi chạy
docker run img                          # entrypoint + CMD mặc định
docker run img nginx -T                 # ghi đè CMD
docker run -it --entrypoint sh img      # ghi đè ENTRYPOINT để debug
```

Ví dụ trình bày: ENTRYPOINT là nghề, CMD là món mặc định

Container là một đầu bếp. **ENTRYPOINT** là nghề của đầu bếp (nấu phở), **CMD** là món mặc định (phở bò). Khách gọi món khác thì chỉ đổi CMD (phở gà), không đổi nghề.

Build context, cache và multi-stage

Build context và .dockerignore

- Context là tập file build được truy cập: thư mục, URL Git, tarball hoặc `-`
- COPY file ngoài context hoặc bị ignore sẽ gây lỗi `failed to compute cache key: ... "/main.c": not found`
- `.dockerignore` đặt ở gốc context, cú pháp như `.gitignore`, giúp build nhanh hơn: `.git` `node_modules` `*.log` `.env`. Podman ưu tiên `.containerignore`

Quy tắc cache

- Mỗi lệnh tạo 1 layer. **Một layer đổi thì mọi layer phía sau build lại**
- ADD/COPY so checksum nội dung file (không tính mtime). RUN chỉ so chuỗi lệnh
- Thứ tự đúng: OS packages → file dependency → cài dependency → mã nguồn
- `--pull` lấy base image mới, `--no-cache` bỏ cache

```
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download        # được cache nếu go.mod không đổi
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app .

FROM build AS test
RUN go test ./...

FROM gcr.io/distroless/static AS final
COPY --from=build /out/app /app
USER 65532
ENTRYPOINT ["/app"]
```

```
docker build --target test -t app:test .   # dừng ở stage test
docker build -t app:1.0 .                  # build tới stage cuối
```

Multi-stage: mỗi FROM là một stage, chỉ chép artifact cần thiết sang stage cuối nên image nhỏ, không chứa compiler. Các stage chạy song song được và dùng lại được. Podman cũng hỗ trợ `--target`.

BuildKit, buildx và best practices

```
# syntax=docker/dockerfile:1
# Cache mount: giữ cache pip giữa các lần build
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
# Secret mount: không lưu vào layer, nằm ở /run/secrets/aws
RUN --mount=type=secret,id=aws \
    AWS_SHARED_CREDENTIALS_FILE=/run/secrets/aws \
    aws s3 cp s3://bucket/model.bin .
```

```
docker build --secret id=aws,src=$HOME/.aws/credentials .
docker buildx build --platform linux/amd64,linux/arm64 \
  -t registry.company.vn/team/app:1.0 --push .
```

BuildKit là builder mặc định. `RUN --mount` có các kiểu bind (mặc định), cache, tmpfs, secret, ssh. Cache mount chỉ để tăng tốc và không xuất qua `--cache-to`, nên CI runner mới có cache rỗng. Multi-platform có 3 cách: QEMU, builder nhiều node native, cross-compile trong multi-stage.

Best practices

1. Base image nhỏ và tin cậy (Official Images, Verified Publisher)
2. Ghim version/digest để tái lập được. Đổi lại, image không tự nhận bản vá
3. Rebuild thường xuyên vì image bất biến
4. Mỗi container một mối quan tâm. Không cài gói thừa vì container là tạm thời
5. Chạy **non-root** với UID/GID cố định. Tránh sudo (dùng gosu nếu cần)
6. Dùng `set -o pipefail`, vì `/bin/sh -c` chỉ xét mã thoát của lệnh cuối

🎯 **Đầu ra:** Viết được Dockerfile multi-stage, tận dụng cache, không lộ bí mật và chạy non-root

Network drivers: default bridge và user-defined bridge

Driver	Đặc điểm
`bridge`	**Mặc định.** Mạng riêng trên host, ra ngoài qua NAT
`host`	Bỏ cô lập, dùng thẳng stack mạng của host
`none`	Cô lập hoàn toàn, không có mạng
`overlay`	Nối nhiều host (Swarm)
`macvlan`	Container có MAC riêng, trông như thiết bị vật lý
`ipvlan`	Giống macvlan nhưng không cấp MAC riêng

```
docker network create -d bridge --subnet 172.20.0.0/24 my-net
docker network ls ; docker network inspect my-net
docker network connect my-net my-nginx     # gắn khi đang chạy
docker network disconnect my-net my-nginx
docker network rm my-net                   # phải disconnect trước
docker port my-nginx                       # xem các port đã publish
```

	Default bridge (`docker0`)	User-defined bridge
Tạo	Tự động. Container không có `--network` sẽ gắn vào	`docker network create`
DNS theo tên	Không	**Có**
Cô lập	Mọi container chung một mạng	Tốt hơn, tách theo ứng dụng
Connect / disconnect	Phải tạo lại container	Được khi đang chạy
Trạng thái	Legacy	**Khuyến nghị**

Container ở 2 bridge khác nhau chỉ nói chuyện được qua port đã publish. Khoảng 1000 container trên một bridge có thể không ổn định. Một container gắn được nhiều network, chọn gateway bằng `gw-priority`. Docker **không** tạo rule firewall cho ipvlan, macvlan, host.

💡 **Ví dụ trình bày: user-defined bridge là mạng văn phòng**

Mỗi ứng dụng một văn phòng riêng, có tổng đài gọi nhau theo **tên** (DNS). **Default bridge** là sảnh chung, ai cũng ngồi chung và phải nhớ số máy lẻ (IP).

Publish port, bảo mật và firewalld

Publish port

- `-p 8080:80` · `-p 192.168.1.100:8080:80` · `-p 8080:80/udp` . `-P` publish mọi port EXPOSE ra port ngẫu nhiên
- Mặc định bind **mọi địa chỉ** host (IPv4 + IPv6), tức là "không an toàn mặc định". Chỉ cho host truy cập: `-p 127.0.0.1:8080:80`
- **Docker 28:** host cùng L2 không còn truy cập được port publish vào localhost (trước 28.0.0 thì được)
- Đổi bind mặc định: `-o com.docker.network.bridge.host_binding_ipv4=127.0.0.1` hoặc `default-network-opts` trong `daemon.json`

DNS nhúng, container gọi container

- Trên user-defined network, container gọi nhau bằng **tên** hoặc IP qua DNS nhúng `127.0.0.11`
- Trong cùng user-defined bridge, **mọi port mở cho nhau**, không cần `-p`

Docker và firewall

- Docker xử lý trong bảng **nat** trước chain **INPUT**, nên port publish **vượt qua ufw**. Với firewalld hiệu ứng tương tự (kiểm chứng trong lab)
- firewalld: Docker tạo zone `docker` (target ACCEPT) và policy `docker-forwarding`
- Backend mặc định là iptables. nftables (`firewall-backend`) còn experimental và không tự bật IP forwarding. Không đặt `"iptables": false`

```
docker run -d --name web -p 127.0.0.1:8080:80 nginx
firewall-cmd --get-active-zones     # có zone docker
nft list ruleset | grep -i docker   # RHEL 9: iptables-nft
docker exec web cat /etc/resolv.conf   # nameserver 127.0.0.11
```

Đầu ra: Kết nối được các container bằng user-defined network và publish port an toàn

Volume, bind mount và tmpfs

Writable layer là tạm thời: file tạo trong container nằm ở writable layer, phía trên các layer chỉ đọc của image. Layer này gắn riêng với từng container và **mất khi container bị xóa**.

Loại	Đặc điểm	Dùng cho
Volume	Daemon quản lý, còn lại sau khi xóa container	Database, dữ liệu bền vững (**khuyến nghị**)
Bind mount	Nối thẳng đường dẫn host. Mặc định ghi được lên host	Mã nguồn khi dev, file cấu hình
tmpfs	Nằm trong RAM, mất khi container dừng	Dữ liệu tạm, nhạy cảm

Cú pháp --mount và -v

- Nên dùng `--mount`: rõ ràng và đủ tùy chọn (volume driver, `volume-subpath`, Swarm)
- `-v` tự tạo thư mục host nếu chưa có. `--mount` báo lỗi, trừ khi thêm `bind-create-src`
- **RHEL/SELinux:** nhãn chỉ đặt được bằng `-v ...:Z` hoặc `:z`
- Volume **rỗng** gắn vào thư mục có dữ liệu thì Docker copy dữ liệu vào (tắt bằng `volume-nocopy`). Bind mount thì **che** dữ liệu sẵn có

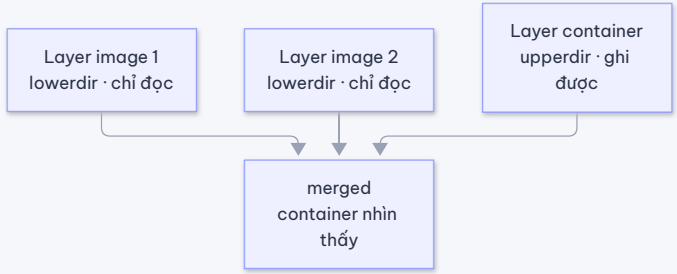
```
docker volume create pgdata
docker run -d --name db --mount \
  type=volume,src=pgdata,dst=/var/lib/postgresql/data postgres:16
docker run -d -v ./conf:/etc/nginx/conf.d:ro,Z nginx
docker run -d --tmpfs /tmp:size=64m app
docker volume ls ; docker volume inspect pgdata
#  "Mountpoint": "/var/lib/docker/volumes/pgdata/_data"
docker volume rm pgdata ; docker volume prune
```

Anonymous volume có tên ngẫu nhiên, vẫn còn sau khi xóa container (trừ khi chạy với `--rm`) và không tự được dùng lại. Volume **không bao giờ tự bị xóa**.

💡 **Ví dụ trình bày: lưu dữ liệu như đồ dùng văn phòng**

Writable layer là bảng nháp, bị xóa khi tan ca. **Volume** là ổ cứng gắn ngoài, đổi máy vẫn còn. **Bind mount** là cửa thông thẳng sang kho của host. **tmpfs** là giấy note trên bàn.

overlay2 và containerd image store




```
docker info | grep -E "Storage Driver|Backing Filesystem"
docker info -f '{{ .DriverStatus }}'
#   io.containerd.snapshotter.v1 nếu dùng containerd store
docker system df -v   # Images · Containers · Volumes · Cache
```

overlay2

- OverlayFS ghép **lowerdir** (layer image, chỉ đọc) và **upperdir** (layer container, ghi được) thành **merged**. Tối đa 128 lower layer
- Sửa file của lowerdir thì file được copy lên upperdir (copy-on-write). Xóa file thì tạo **whiteout**, file gốc vẫn nằm ở lowerdir
- **RHEL:** xfs phải có `ftype=1`. Không sửa tay `/var/lib/docker`

containerd image store

- Từ **Docker Engine 29**, đây là mặc định cho bản **cài mới**. Bản nâng cấp vẫn chạy overlay2
- Bật thủ công trong `/etc/docker/daemon.json` : `"features": {"containerd-snapshotter": true}`
- Tốn đĩa hơn (giữ cả layer nén lẫn đã giải nén), lưu ở thư mục riêng (thường `/var/lib/containerd`), không dùng được với `userns-remap`

🎯 **Đầu ra:** Lưu được dữ liệu bền vững bằng volume và giải thích được cơ chế overlay2

Docker Compose: v2 và file compose.yaml

	Compose v1	Compose v2 / v5
Lệnh	`docker-compose`	`docker compose` (plugin)
Viết bằng	Python	Go (v5 có thêm Go SDK)
`version:`	2.0 – 3.8	Bỏ qua, đã lỗi thời
Hỗ trợ	Hết sau 6/2023	Đang hỗ trợ

Các phần tử chính

- **services · networks · volumes · configs · secrets**. Configs và secrets gắn vào container dưới dạng **file**
- `depends_on` có điều kiện `service_started`, `service_healthy`, `service_completed_successfully`. `restart: true` khởi động lại theo dependency
- `$$` để container tự đánh giá biến, Compose không nội suy
- `.env` nội suy biến **trong file Compose**, `env_file` đưa biến **vào container**
- Project name: `name:`, `-p` hoặc `COMPOSE_PROJECT_NAME`

Tên file nên dùng `compose.yaml`. Cảnh báo "version is obsolete" thì xóa dòng `version:`. RHEL 9 cài gói `docker-compose-plugin`.

```yaml
name: shop
services:
  web:
    image: registry.company.vn/team/webapp:1.4.2
    ports: ["8080:3000"]
    environment: { DB_HOST: db }
    depends_on:
      db: { condition: service_healthy, restart: true }
    deploy:
      resources: { limits: { cpus: "0.5", memory: 512M } }
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_pass
    secrets: [db_pass]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER:-postgres}"]
      interval: 10s
      retries: 5
volumes:
  pgdata: {}
secrets:
  db_pass: { file: ./db_pass.txt }
```

Lệnh Compose, mạng mặc định và Podman Quadlet

```
docker compose up -d           # tạo network, volume, chạy nền
docker compose ps ; docker compose logs -f web
docker compose exec web sh     # vào shell của service web
docker compose config          # file đã hợp nhất + nội suy biến
docker compose down            # xóa container + network
docker compose down -v         # xóa luôn named volume ⚠
docker compose watch           # đồng bộ code khi dev
```

Mạng mặc định

- Compose tạo mạng `<project>_default` (bridge). Mỗi service có tên trong DNS nội bộ
- Gọi nhau bằng **tên service** và **port trong container** (`db:5432`), không dùng port host
- IP đổi khi container được tạo lại, nên luôn dùng tên. `network_mode: host` làm mất DNS theo tên service

💡 **Compose là bản phối cho cả dàn nhạc**

Một file mô tả từng nhạc công (**service**), sân khấu (**network**) và nhạc cụ riêng (**volume**). `up` là ra hiệu bắt đầu, `down` là thu dọn.

Podman trên RHEL 9

- `podman compose` chỉ là lớp vỏ gọi provider bên ngoài (ưu tiên docker-compose nếu có). Đổi qua `compose_providers` hoặc `PODMAN_COMPOSE_PROVIDER`
- **Quadlet** (Podman ≥ 4.6): chạy container bằng **systemd**, cách Red Hat hướng dẫn

```
# ~/.config/containers/systemd/web.container
#   [Container]
#   Image=registry.company.vn/team/webapp:1.4.2
#   PublishPort=8080:3000
#   Volume=webdata:/data:Z
#   [Install]
#   WantedBy=default.target
systemctl --user daemon-reload  # generator sinh ra web.service
systemctl --user start web      # container tên systemd-web
loginctl enable-linger $USER    # chạy cả khi user không đăng nhập
```

🎯 **Đầu ra:** Dựng được ứng dụng nhiều service bằng Compose và chạy container bằng systemd (Quadlet)

Cảm ơn anh/chị đã lắng nghe

Kế hoạch tiếp theo: .....

Hỏi & Đáp

Lê Tấn Lực · Đỗ Hoàng Hiếu